

Structures discrètes et algorithmes – INF 101

Durée : 1 h 30

Calculatrices et ordinateurs interdits.

Documents autorisés : deux feuilles recto verso (quatre pages) format A4.

L'épreuve est constituée de trois exercices indépendants.

Le barème n'est donné qu'à titre indicatif et pourra être modifié.

Sauf mention contraire dans l'énoncé, une réponse non justifiée pourra ne pas être considérée comme juste.

On pourra utiliser les graphes figurant en annexe et remettre celle-ci avec la copie.

Le corrigé de l'épreuve est disponible sur le site pédagogique de SDA – INF 101.

Exercice 1 (4 points)

On considère une suite triée de n entiers distincts. Soit x un des entiers de cette suite (mais on ne connaît pas la position de x dans la suite).

1. (2 points) Décrire le principe d'une méthode permettant de supprimer x de la suite avec une plus petite complexité dans le pire des cas pour chacune des deux situations suivantes ; on précisera la complexité des méthodes proposées.

a. La suite est représentée par un tableau (trié) T dont les cases sont numérotées de 1 à n et on veut conserver le fait que T est trié (et sans case vide) après suppression de x .

Corrigé

On commence par localiser la position p de x dans T . Pour cela, on procède par dichotomie, ce qui est de complexité $O(\log n)$. Puis, pour i variant de $p + 1$ à n , on recopie $T[i]$ dans $T[i - 1]$. À la fin de la boucle, T contient les données initiales triées, sauf x , dans les cases d'indices compris entre 1 et $n - 1$. La complexité de la boucle est en $O(n)$. On termine le procédé en décrémentant n , ce qui se fait en $O(1)$. La complexité totale est donc en $O(n)$.

Fin de corrigé

b. La suite est représentée par une liste chaînée L et on veut conserver le fait que L est triée après suppression de x .

Corrigé

On traite à part le cas pour lequel x figure en début de liste. Dans ce cas, on élimine x en décalant d'un maillon le pointeur donnant accès à L , ce qui se fait en $O(1)$.

Sinon, pour localiser le maillon de L qui contient x , on se déplace le long de L de maillon en maillon, à l'aide de deux pointeurs qui se suivent p et q , q étant en retard d'un maillon par rapport à p : autrement dit, le maillon pointé par p est le successeur de celui pointé par q dans L . On localise ainsi x avec une complexité en $O(n)$. Quand p pointe sur le maillon contenant x , q pointe sur le maillon précédent. On modifie alors le chaînage pour que le successeur du maillon pointé par q soit le maillon successeur du maillon pointé par p (en pseudo-code et avec des notations classiques, cela s'écrit : $(q \rightarrow \text{suivant}) \leftarrow (p \rightarrow \text{suivant})$, où le signe « $\leftarrow$ » représente l'affectation ; en Java, cela s'écrit : $q.\text{suivant} = p.\text{suivant}$), ce qui se fait en $O(1)$. Au total, la suppression de x en gardant une liste triée se fait en $O(n)$.

Remarque : si on n'a plus besoin de x , on peut libérer la place occupée par le maillon associé à x en mémoire, ce qui ne change pas la complexité ci-dessus.

Fin de corrigé

2. (2 points) Pour chacun des deux cas précédents, peut-on obtenir une meilleure complexité si on n'exige plus de conserver une suite triée après suppression de x ? Si oui, indiquer les modifications à apporter aux réponses précédentes et la nouvelle complexité obtenue.

Corrigé

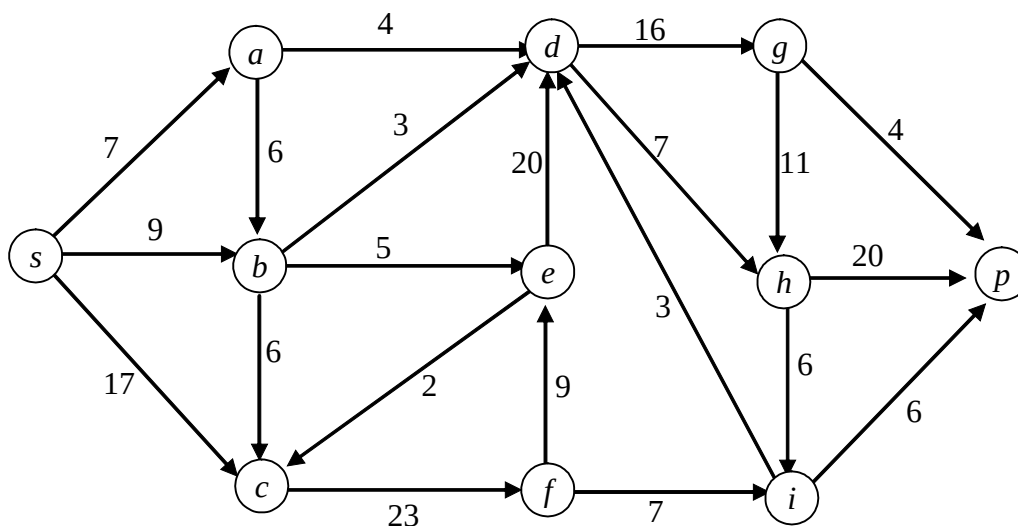
On peut améliorer la complexité dans le cas d'une représentation à l'aide d'un tableau. Comme dans la question 1, on localise la position p de x dans T par dichotomie, en $O(\log n)$. Puis on recopie $T[n]$ dans $T[p]$, ce qui se fait en $O(1)$. On termine le procédé en décrémentant n , ce qui se fait encore en $O(1)$. La complexité totale est donc maintenant en $O(\log n)$.

En revanche, la localisation de x dans la liste chaînée en $O(n)$ subsiste même si on ne garde pas le caractère trié de L . Relâcher le caractère trié de L ne change donc pas la complexité qui reste en $O(n)$.

Fin de corrigé

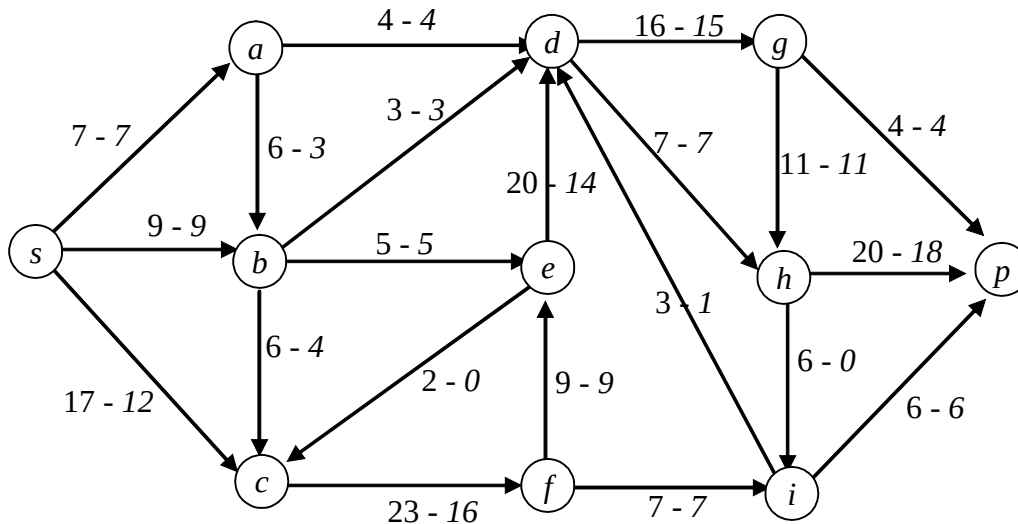
Exercice 2 (6 points)

1. (3 points) Déterminer une coupe séparant s de p de capacité minimum dans le réseau ci-dessous, où les entiers inscrits à côté des arcs indiquent les capacités de ces arcs. Indiquer la méthode utilisée pour résoudre ce problème (il n'est pas demandé de préciser les calculs intermédiaires permettant d'obtenir la coupe) et prouver que la coupe proposée est bien de capacité minimum.



Corrigé

On détermine un flot maximum à l'aide de l'algorithme de Ford et Fulkerson. On part par exemple du flot donné par les valeurs écrites en italiques ci-dessous :



L'algorithme de marquage permet alors de marquer les sommets suivants :

initialisation : s avec la marque $(., +\infty)$;

examen de s : c avec la marque $(s, +5)$;

examen de c : b avec la marque $(c, -4)$; f avec la marque $(c, +4)$;

examen de b : a avec la marque $(b, -3)$;

examen de f : rien ;

examen de a : rien.

On ne peut pas atteindre p : le flot actuel est de valeur maximum. On en déduit une coupe de capacité minimum en isolant les sommets marqués (s, a, b, c, f) des autres (d, e, g, h, i, p). On vérifie que la capacité de cette coupe est bien égale à la valeur du flot précédent : 28, ce qui établit l'optimalité de la coupe proposée (et du flot ci-dessus).

Fin de corrigé

2. (3 points) On a la possibilité d'augmenter la capacité de certains arcs d'une unité. Indiquer pour les deux cas suivants si une telle augmentation change la valeur de la capacité minimum des coupes. On justifiera les réponses et, si la capacité minimum change, on indiquera la nouvelle valeur obtenue.

a. La capacité de l'arc (d, h) augmente d'une unité.

Corrigé

Le sommet d n'étant pas accessible dans le marquage précédent, augmenter la capacité de l'arc (d, h) ne change rien : la coupe précédente reste de capacité minimum.

Fin de corrigé

b. La capacité de l'arc (a, d) augmente d'une unité.

Corrigé

L'arc (a, d) étant saturé alors que a était marqué précédemment, augmenter la capacité de l'arc (a, d) peut changer la situation. Pour savoir si c'est le cas, on reprend l'algorithme de marquage. La différence avec ce qui précède se situe au niveau de l'examen de a : on peut désormais marquer d . On obtient successivement :

examen de a : d avec la marque $(a, +1)$;

examen de d : g avec la marque $(d, +1)$; i avec la marque $(d, -1)$;

examen de g : rien ;

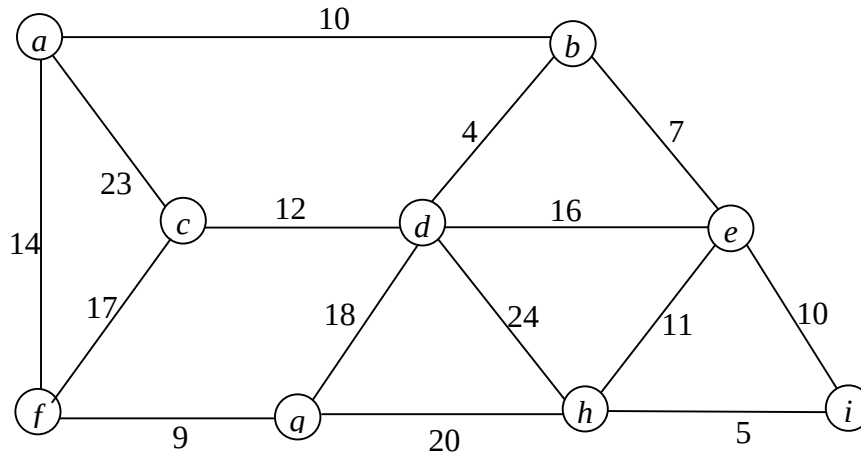
examen de i : rien.

On ne peut encore pas atteindre p : la valeur maximum des flots reste égale à 28 et donc la capacité minimum des coupes reste aussi égale à 28, même si la coupe précédente n'est plus de capacité minimum (mais la coupe séparant s, a, b, c, f, g, i de d, e, h, p est bien de capacité 28).

Fin de corrigé

Exercice 3 (10 points)

1. (4 points) En adaptant un algorithme vu en cours que l'on identifiera, déterminer un arbre couvrant de poids **maximum** T_0 dans le graphe G_0 ci-dessous. On précisera et justifiera les adaptations effectuées pour l'algorithme. On donnera les indications nécessaires pour convaincre le correcteur que l'arbre est bien obtenu à l'aide de l'algorithme et non deviné.



Corrigé

On adapte l'algorithme de Kruskal ou l'algorithme de Prim de la façon suivante.

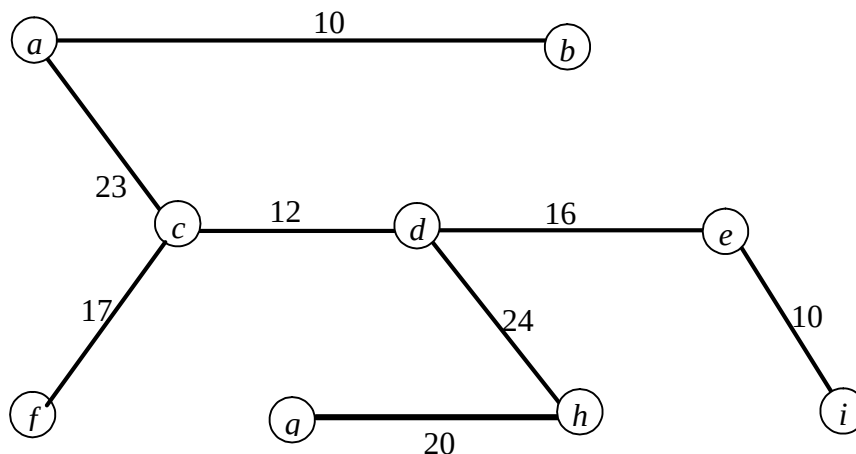
* Pour l'algorithme de Kruskal : on trie initialement les arêtes selon les poids décroissants ; le reste est inchangé.

* Pour l'algorithme de Prim, en reprenant les notations du polycopié : pour tout sommet x autre que x_0 , les attributs $d(x)$ sont initialisés à $-\infty$ (au lieu d'être initialisés à $+\infty$) ; le test « si $p(\text{pivot}, z) < d(z)$ » devient « si $p(\text{pivot}, z) > d(z)$ » ; le sommet *pivot* est un sommet non dans S qui maximise d .

Ces adaptations reposent sur le résultat selon lequel le maximum d'une fonction f sur un ensemble X est l'opposé du minimum de $-f$ sur le même ensemble X . Il suffirait donc de changer le signe des poids du graphe pour se ramener à la recherche d'un arbre couvrant de poids minimum. Pour l'algorithme de Kruskal, l'examen des arêtes selon les opposés des poids croissants est équivalent à l'examen des arêtes selon les poids initiaux décroissants ; d'où l'adaptation. Pour l'algorithme de Prim, les modifications proposées résultent aussi de la prise en compte de l'opposé des poids. En effet, si on pose $p' = -p$ et $d' = -d$, on est amené à chercher un arbre de poids minimum pour p' et on a alors les équivalences suivantes :

- initialiser les attributs $d'(x)$ à $+\infty \Leftrightarrow$ initialiser les attributs $d(x)$ à $-\infty$;
- $p'(\text{pivot}, z) < d'(z) \Leftrightarrow p(\text{pivot}, z) > d(z)$
- *pivot* est un sommet qui minimise $d' \Leftrightarrow$ *pivot* est un sommet qui maximise d .

L'application de l'un de ces algorithmes adaptés donne l'arbre suivant :



Pour l'algorithme de Kruskal, les arêtes ont été sélectionnées dans l'ordre suivant : $\{d, h\}$, $\{a, c\}$, $\{g, h\}$ (on rejette alors $\{d, g\}$ qui formerait un cycle avec les arêtes $\{d, h\}$ et $\{g, h\}$), $\{c, f\}$, $\{d, e\}$ (on rejette alors $\{a, f\}$ qui formerait un cycle avec les arêtes $\{a, c\}$ et $\{c, f\}$), $\{d, c\}$ (on rejette alors $\{e, h\}$ qui formerait un cycle avec les arêtes $\{d, h\}$ et $\{d, e\}$), $\{a, b\}$ et $\{e, i\}$ (les deux dernières arêtes pouvant être interverties) ; on s'arrête alors, puisque l'on a sélectionné huit arêtes alors qu'il y a neuf sommets.

Pour l'algorithme de Prim, si on part du sommet a comme premier pivot, on atteint successivement les pivots suivants et on sélectionne successivement les arêtes $\{pivot, proche(pivot)\}$ suivantes : c avec $\{c, a\}$, d avec $\{d, c\}$, h avec $\{h, d\}$, g avec $\{g, h\}$, e avec $\{e, d\}$, b avec $\{b, a\}$, i avec $\{i, e\}$ (les deux dernières étapes pouvant être interverties).

Fin de corrigé

On considère maintenant un graphe $G = (X, A)$ non orienté, connexe, dont les arêtes sont pondérées par une fonction p à valeurs strictement positives. Pour tout sous-ensemble B de A , on note H_B le graphe partiel de G obtenu par suppression des arêtes de B : $H_B = (X, A - B)$. Par ailleurs, la somme des poids $p(b)$ des arêtes b appartenant à B est appelée *poids de B* et est notée $p(B)$:
$$p(B) = \sum_{b \in B} p(b).$$

Un sous-ensemble B d'arêtes de G s'appelle une *base de cycles de G* si B contient au moins une arête de tout cycle de G . On cherche à déterminer une base de cycles de poids minimum par rapport à la fonction p .

2. (1 point) Montrer que B est une base de cycles si et seulement si H_B est sans cycle.

Corrigé

Supposons d'abord que B est une base de cycles et supposons néanmoins que H_B contienne un cycle C . Par définition de H_B , aucune arête de C ne serait dans B , contradiction avec le fait que B est une base de cycle. Donc H_B est sans cycle.

Réciproquement, supposons que B ne soit pas une base de cycles. Il existe alors un cycle C dont aucune arête ne soit dans B et C est alors un cycle de H_B , qui ne peut donc être sans cycle.

Fin de corrigé

3. (1 point) Soit B une base de cycles de poids minimum. Montrer que H_B est connexe.

Corrigé

Supposons H_B non connexe. Comme G est connexe, il existe deux composantes connexes de H_B reliées par une arête dans G . Soit $\{x, y\}$ une telle arête de G (donc avec x et y appartenant à des composantes connexes distinctes de H_B). L'arête $\{x, y\}$ n'appartenant pas à H_B , elle appartient à B . Posons $B' = B - \{x, y\}$. On a alors $H_{B'} = H_B \cup \{x, y\}$; $H_{B'}$ est donc sans cycle et, d'après la question 2, B' est une base de cycles. Or, on a $p(B') = p(B) - p(\{x, y\})$. Les poids des arêtes étant supposés strictement positifs, il vient $p(B') < p(B)$, contradiction avec le fait que B est de poids minimum. Donc H_B est connexe.

Fin de corrigé

4. (1,5 points) En s'appuyant sur ce qui précède, concevoir un algorithme permettant de déterminer une base de cycles de poids minimum. On indiquera succinctement pourquoi cet algorithme convient.

Corrigé

Les questions 2 et 3 montrent que, si B est une base de cycle de poids minimum, alors H_B est un arbre. De plus, on a : $p(B) + p(A - B) = p(A)$. Chercher B de poids minimum revient à chercher H_B de sorte que $p(A - B)$ soit maximum : H_B est donc un arbre couvrant G de poids maximum par rapport à p .

On en déduit l'algorithme suivant pour obtenir une base de cycles de poids minimum :

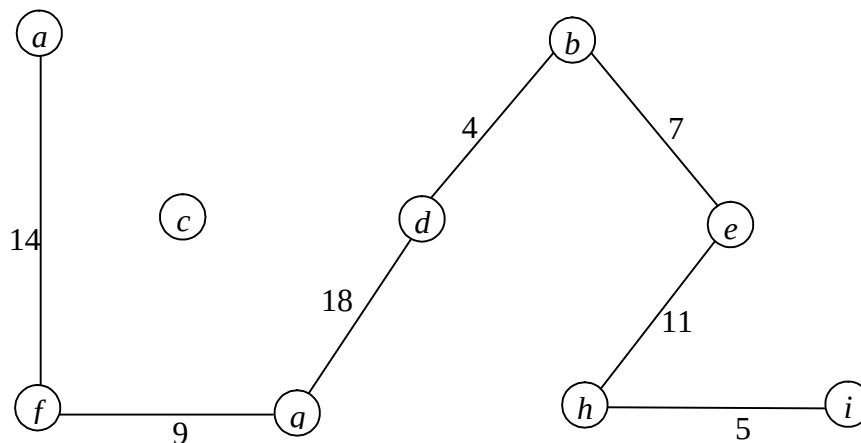
- déterminer un arbre couvrant G de poids maximum par rapport à p ; soit T les arêtes de cet arbre ;
- poser $B = A - T$.

Fin de corrigé

5. (0,5 point) Déterminer une base de cycles de poids minimum pour le graphe G_0 ci-dessus.

Corrigé

L'application de ce qui précède à G_0 donne, compte tenu du résultat de la question 1, la base de cycles suivantes : $B = \{\{a, f\}, \{f, g\}, \{d, b\}, \{g, d\}, \{b, e\}, \{e, h\}, \{i, h\}\}$, de poids 68. Le graphe partiel engendré par B est dessiné ci-dessous.



Fin de corrigé

6. (2 points) Écrire le problème de décision, noté BC , associé à la recherche d'une base de cycles de poids minimum. On se place dans l'hypothèse $P \neq NP$. Le problème BC appartient-il à la classe P ? à la classe NP ? est-il NP -complet ? Justifier les réponses.

Corrigé

Le problème BC est le suivant :

Nom : BC (base de cycles)

Données : un graphe $G = (X, A)$ non orienté, connexe, dont les arêtes sont pondérées par une fonction p à valeurs strictement positives ; un entier K ;

Question : existe-t-il $B \subseteq A$ tel que B soit une base de cycles de G avec $p(B) \leq K$?

Le problème BC appartient à la classe P . En effet, l'algorithme proposé dans la question 4 pour déterminer une base de cycles de poids minimum est de même complexité que celui permettant de déterminer un arbre couvrant de poids minimum (via l'adaptation de l'algorithme de Kruskal ou celui de Prim), lequel est polynomial par rapport à la taille des données (au moins de l'ordre de n puisque que le graphe est connexe, alors que l'algorithme de Kruskal est en $O(n^2 \log n)$ et celui de Prim en $O(n^2)$).

Comme la classe P est incluse dans la classe NP , on obtient que BC appartient à NP .

Enfin, sous l'hypothèse $P \neq NP$, on sait qu'aucun problème de P n'est NP -complet. Par conséquent, toujours avec cette hypothèse, BC n'est pas NP -complet.

Fin de corrigé